Linux C++ 安装和使用grpc和jsoncpp库

Posted on 2023-07-30 21:16:34 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2TIG572uTKxQxned7LCk8KoulfL)

简介

本文主要教会大家如何在Linux环境搭建C++ 所需的grpc和jsoncpp库，并教会大家如何编写cmake，并配置使用这些库。

解决vim乱码

为了解决Linux环境下打开vim中文乱码的问题
用vim打开用户目录下的vim配置文件

```
1. vim ~/.vimrc
```

配置如下

```
1. set termencoding=utf-8
2. set encoding=utf8
3. set fileencodings=utf8,ucs-bom,gbk,cp936,gb2312,gb18030
```

配置和使用jsoncpp

如果你是ubuntu系统，可以通过如下命令直接安装

```
1. 可以通过指令apt install  libjsoncpp-dev 安装
```

但是如果是其他Linux系统最好是手动安装源码包，我的操作都是在ubuntu为基础镜像生成的docker中进行的，如果大家使用的是宿主机，可以直接安装。
推荐源码安装，去github下载

https://github.com/open-source-parsers/jsoncpp/releases (https://github.com/open-source-

parsers/jsoncpp/releases)

可以选择用wget 命令
我在电脑下好后传到云服务器上
然后在云服务器上copy到docker里, 如果你是在宿主机进行的，可以略去这一步。

```
1. docker cp /home/ubuntu/download/jsoncpp-1.9.5.tar.gz  cppubuntu:/test
```

进入容器

```
1. docker exec -it cppubuntu /bin/bash
```

接下来解压压缩包，无论docker还是宿主机内，都需执行如下命令

```
1. tar zxvf ./jsoncpp-1.9.5.tar.gz
```

进入到源码目录

```
1. cd  ./json
```

创建目录

```
1. mkdir build
```

进入目录

```
1. cd build
```

执行cmake生成makefile

```
1. cmake ../
```

执行make

```
1. make
```

执行安装

```
make install
```

更新库

```
ldconfig
```

写一个jsoncpp的测试cpp

```
#include<json/json.h>
#include<iostream>
using namespace std;

int main(int argc, char** argv)
{
Json::Value root;
Json::FastWriter fast;
root["ModuleType"]= Json::Value("你好");
root["ModuleCode"]= Json::Value("22");
root["ModuleDesc"]= Json::Value("33");
root["DateTime"]= Json::Value("44");
root["LogType"]= Json::Value("55");
cout<<fast.write(root)<<endl;
return 0;
}
```

执行 编译

```
g++ jsontest.cpp  -o jsontest -ljsoncpp
```

或者写个cmake

```
cmake_minimum_required(VERSION 3.12)
project(jsontest)

# 设置 C++ 标准
set(CMAKE_CXX_STANDARD 17)

# 添加可执行文件和源文件
file(GLOB SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/*.cpp)
set(JSONCPP_INC_DIR  /usr/local/include)

add_executable(jsontest ${SOURCES})

# 包含头文件路径（包括其他目录）
target_include_directories(jsontest
    PRIVATE
    ${JSONCPP_INC_DIR}
)

# 链接 Boost 库
target_link_libraries(jsontest PRIVATE jsoncpp)
```

运行 `./jsontest`
输出

```
{"DateTime":"44","LogType":"55","ModuleCode":"22","ModuleDesc":"33","ModuleType":"\u4f60\u597d"}
```

grpc配置和使用

克隆grpc指定分支

```
git clone -b v1.34.0 https://gitee.com/mirrors/grpc-framework.git grpc
```

进入目录并更新子模块

```
cd grpc
git submodule update --init
```

编译并生成grpc库

```
cd grpc
mkdir build
cd build
// 指定安装路径 /usr/local
cmake -DCMAKE_INSTALL_PREFIX=/usr/local ..
make -j2
sudo make install
```

测试安装成功与否

编译源代码中的helloworld文件夹下的文件，步骤如下：

```
#进入grpc文件夹下
cd examples/cpp/helloworld
mkdir build
cd build
# 编译
cmake ..
make -j8
```

编译完成后，分别执行greeter_server和greeter_client即可测试。

项目应用grpc

我们的项目中也用到了grpc， 需要编写一个CMakeLists.txt 配置grpc。
大家可以克隆我的boost项目代码

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

进入day19-Grpc-Server目录，我们编写如下的CMakeLists.txt

```
cmake_minimum_required(VERSION 3.1)

project(GrpcServer LANGUAGES CXX)

set(CMAKE_INCLUDE_CURRENT_DIR ON)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

#假设已经安装好grpc了
find_package(Threads REQUIRED)

set(protobuf_MODULE_COMPATIBLE TRUE)
find_package(Protobuf CONFIG REQUIRED)
message(STATUS "Using protobuf ${Protobuf_VERSION}")

set(_PROTOBUF_LIBPROTOBUF protobuf::libprotobuf)
set(_REFLECTION gRPC::grpc++_reflection)


# Find gRPC installation
# Looks for gRPCConfig.cmake file installed by gRPC's cmake installation.
find_package(gRPC CONFIG REQUIRED)
message(STATUS "Using gRPC ${gRPC_VERSION}")

set(_GRPC_GRPCPP gRPC::grpc++)

# 添加可执行文件和源文件
file(GLOB SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/*.cpp)
file(GLOB PBSOURCES ${CMAKE_CURRENT_SOURCE_DIR}/*.cc)


add_executable(GrpcServer ${SOURCES}
        ${PBSOURCES})

target_link_libraries(GrpcServer
    ${_REFLECTION}
    ${_GRPC_GRPCPP}
    ${_PROTOBUF_LIBPROTOBUF})
```

我们新建一个build文件夹

```
1. mkdir build
```

进入build文件夹里，执行 `cmake ..` , 再执行 `make` 即可。
直接执行 `GrpcServer` 就可以看到我们的程序跑起来了。

总结

本文介绍了如何使用Linux环境下配置和使用grpc和cppjson库，我将docker打包为镜像提交到了网盘上，大家感兴趣可以下载看看

链接 (https://pan.baidu.com/s/1K-NGEh88qUMVJexY6SNA5Q?pwd=5wng)
提取码：5wng

浏览(2948) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(541) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)

博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站

个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)